Mew: Minimal Table Retrieval over Data Lakes for Question Answering

Grace Fan
Megagon Labs
grace@megagon.ai

Nima Shahbazi
Megagon Labs
nima@megagon.ai

Aaron Feng
Megagon Labs
aaron@megagon.ai

Nikita Bhutani
Megagon Labs
nikita@megagon.ai

Estevam Hruschka
Megagon Labs
estevam@megagon.ai

ABSTRACT

Answering questions over data lakes requires retrieving relevant tables before a text-to-SQL system can use them. Recent methods consider question relevance and relationships among tables, sometimes using large language models (LLMs), to find relevant tables. However, these methods do not explicitly measure whether the selected tables cover the attributes and values needed to answer a question. This is challenging when tables express those attributes and values differently from the question, or when connecting tables requires bridge tables that provide no direct answer evidence. Retrieval may therefore miss necessary tables or include irrelevant ones. We formalize minimal table retrieval (MTR), the problem of finding the smallest connected set of tables that covers the information needed to answer a question. We show that MTR is NP-complete and present Mew, which represents each question as an *answer-table sketch* of required attributes and value constraints. Mew grounds the sketch in sample rows from the data lake, then uses a join-aware greedy algorithm to select tables that cover the sketch and bridge tables that connect them. The algorithm stops when the sketch is fully covered, without requiring a predefined retrieval cutoff. We evaluate on six benchmarks from BIRD and Spider to model data-lake settings, with up to 16K tables and 100M rows. On these benchmarks, Mew improves table-retrieval F1 by up to 88.2% over existing methods, while improving downstream execution accuracy by up to 96.4% compared to the best-performing baseline.

PVLDB Reference Format:
Grace Fan, Nima Shahbazi, Aaron Feng, Nikita Bhutani, and Estevam Hruschka. Mew: Minimal Table Retrieval over Data Lakes for Question Answering . PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/megagonlabs/mew.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

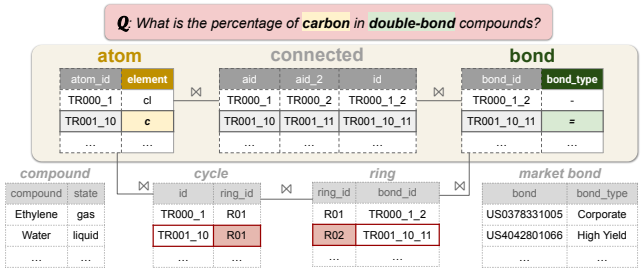


Figure 1: An example of a question Q over a data lake. The shaded top tables `atom`, `connected`, and `bond` answer Q. `compound` and `market bond` on the bottom row are irrelevant tables from the data lake, and `cycle` and `ring` form a longer join path between `atom` and `bond` that gives incorrect results.

1 INTRODUCTION

Table question-answering systems translate natural-language questions into SQL queries over tabular data [43, 72]. Many such systems assume that the database or tables to query are known in advance. In data lakes, however, tables are collected from many sources. Governments and organizations publish tables in open data portals, enterprises pool tables from many sources, and the web contains millions of tables [26, 49]. Users may therefore need to search thousands of tables to find those needed to answer a question. This is especially challenging when the answer spans across several tables [45, 67].

Finding the required tables is difficult because data lakes often lack consistent schemas, complete metadata, and declared primary and foreign keys [26, 49]. Relevant information may be split across tables whose names or values differ from the question, while additional bridge tables may be needed to join them. Users also cannot know in advance how many tables an answer requires. Retrieving too few can omit necessary information, while retrieving too many increases processing costs and can burden downstream text-to-SQL generation.

Example 1. Consider the question Q in Figure 1: "*What is the percentage of carbon in double-bond compounds?*" (adapted from the BIRD benchmark [43]) posed over a data lake of ~13K tables drawn from open data and the web [45]. Here, lexical matches can be misleading. Keyword 'compound' can match the `compound` table from the same domain, while 'bond' can match a `market bond` table from a different domain. The relevant information instead resides

Table 1: Answer accuracy and prompt size of a text-to-SQL model (GPT-5.4-mini) on BIRD questions [43] when 8 or 32 distractor tables, sampled randomly from 13K tables, are added to the gold tables.

Tables	Answer Accuracy	Prompt Tokens
Gold Tables	53.9%	5K
Gold + 8	49.6%	18.5K
Gold + 32	45.7%	64.7K

in `atom` and `bond` (top of Figure 1), where carbon and double bonds are encoded as `'c'` in `atom.element` and `'='` in `bond.bond_type`, respectively. However, these tables cannot be directly joined. The table `connected` links atoms to bonds and serves as a *bridge table*, which contains neither a question-relevant attribute nor value, but is necessary to combine `atom` and `bond`. Although a longer join path through `cycle` and `ring` exists, the join would be on `ring_id`, which misses the carbon atom `TR001_10` and its double bond `TR001_10_11`, since they belong to different rings. Instead, this join connects `TR001_10` with single bond `TR000_1_2`, a bond it is not part of. Every intermediate table on a join path adds a join, possibly a many-to-many join, that can miss correct rows or return incorrect ones. We therefore prefer join paths with fewer intermediate tables [5, 29]. The minimal set of tables that correctly answers Q is thus `atom`, `connected`, and `bond`.

This example illustrates three requirements for effective table retrieval from data lakes. The retrieved tables must *cover* the attributes and values needed to fully answer the question. They must be *connected* so that a downstream SQL query can combine them, possibly through bridge tables. Finally, the selected set should be *minimal* to prevent unnecessary tables from distracting the text-to-SQL model [57], and to lower the cost of downstream text-to-SQL, allowing smaller, cheaper models that are more sensitive to irrelevant tables to generate accurate queries (Section 5.6). This is particularly challenging because data lakes contain many noisy, irrelevant tables, such as `market bond` in Example 1. In a controlled study on an expanded version of BIRD [43] containing 13K tables, adding 32 irrelevant data lake tables to the gold tables lowers the accuracy of a downstream text-to-SQL model from 53.9% to 45.7% and inflates its prompt by 13 times (Table 1). An effective retrieval method must, therefore, satisfy these three requirements to provide a compact, sufficient set of tables for downstream query generation.

Existing table retrieval methods for question answering address different aspects of this problem. Methods typically rank tables by topical similarity to the question [4, 28] or retrieve tables containing cells likely to answer it [24, 64]. Recent methods for multi-table questions additionally account for joinability by decomposing questions into sub-questions, expanding retrieved sets with joinable tables, or selecting joinable subsets of candidates [2, 10, 45, 59]. Most methods return a fixed number of tables [2, 10, 45], while some adaptive retrieval methods adjust this number based on the question [36]. However, these methods do not necessarily ensure that the selected tables jointly cover all required information while remaining connected and minimal. Table retrieval must establish when sufficient information has been covered and which tables are needed to connect it.

Our approach. We present Mew, a method for *minimal table retrieval* (MTR), which aims to find the smallest connected set of tables that covers a question. Instead of matching tables directly against the question text, Mew represents the question as an *answer-table sketch*: a single table specifying the attributes and value constraints required in the answer. Mew grounds the sketch in sample rows of data lake tables, allowing sketch attributes and constraints to match table columns and values even when their representations differ from the question's wording. The sketch also provides an explicit criterion for determining whether the selected tables cover the question. Using this criterion, a join-aware greedy algorithm selects tables that cover sketch elements not yet covered or connect necessary tables through bridge tables. It stops once the sketch is fully covered, without requiring a predefined number of output tables. We show that MTR is NP-complete (Theorem 3), and that the join-aware greedy algorithm adds at most $O(\rho \log m)$ times as many tables as needed to cover the sketch, where m is the number of sketch elements and ρ bounds the number of tables on a join path (Theorem 6).

We make the following contributions.

- We formalize the problem of minimal table retrieval (MTR), which aims to retrieve the smallest connected set of tables that covers the answer table sketch of a question, and show that it is NP-complete.
- We propose *answer table sketches*, which represent a question by the attributes and constraints of the table that answers it. These sketches are refined using column names and values of data lake tables.
- We present a join-aware greedy algorithm that covers the sketch with a connected set of tables, adds the bridge tables on the join paths between them, and achieves an $O(\rho \log m)$-approximation for MTR.
- We evaluate Mew on six benchmarks built from BIRD [43] and SPIDER [72], containing up to 16K tables and 100M rows. We show that Mew outperforms baseline methods by up to 88.2% in F1-score, while improving the execution accuracy of downstream text-to-SQL models by up to 96.4% compared to the best-performing baseline.

2 RELATED WORK

We discuss related work on question-answering (Section 2.1), including multi-table retrieval and text-to-SQL, as well as related work on table discovery in data lakes (Section 2.2).

2.1 Question Answering

Table Retrieval. Many questions in text-to-SQL benchmarks such as BIRD [43] and Spider [72] join several tables of a database. MMQA [67] targets questions that need multi-hop reasoning across tables, and LakeQA [63] poses exploratory questions over a million-scale lake. Join-aware retrieval methods model how tables connect and how relevant each table is to the question. JAR [10] decomposes the question into concepts and attributes, and selects exactly the top tables with a mixed-integer program that rewards both relevance and join compatibility. Similarly, ARM [9] selects connected

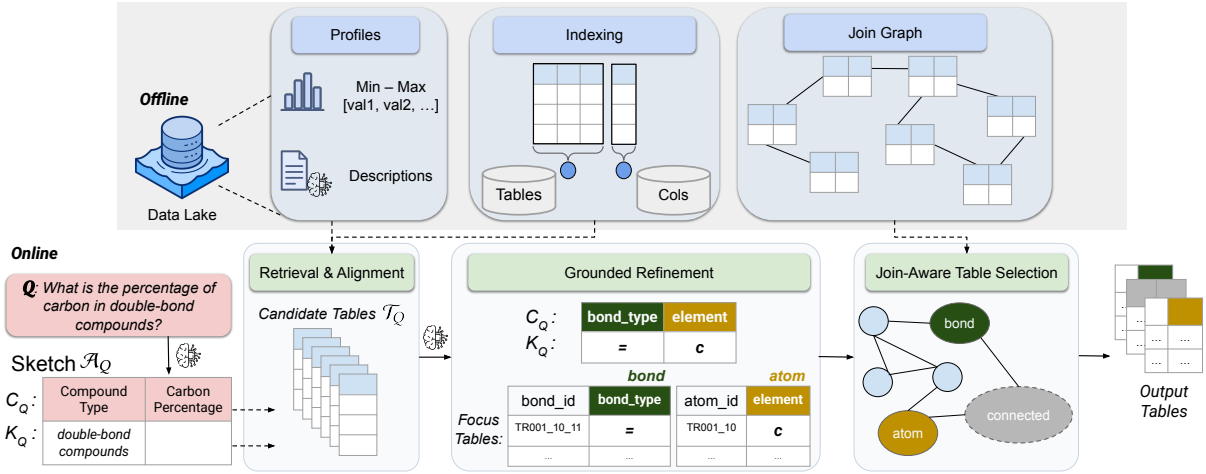


Figure 2: Pipeline of Mew on the question of Example 1. During the offline phase, Mew profiles and indexes the tables of the data lake and builds a join graph (Section 4.1). During the online phase, given a natural-language question, an LLM generates an answer-table sketch from the question, and Mew retrieves candidate tables and aligns them with the sketch (Section 4.2.1). The LLM then grounds the sketch in sample rows of the candidates and returns tables to focus on (Section 4.2.2). Finally, join-aware table selection connects the focus tables through bridge table(s) and returns a small connected set of relevant tables (Section 4.2.3).

tables and passages with a mixed-integer program, and lets an LLM verify the selection. MURRE [75] retrieves tables over multiple hops. DMRAL [45] decomposes the question into sub-questions, and joins relevant tables to each sub-question. REAR [2] uses a dense retriever to retrieve the most similar tables to the question, and expands the table set with joinable tables. Other recent methods leverage an LLM to select relevant tables. CORE-T [59] uses an LLM to select a connected subset of the top-k candidates that are found via dense retrieval. EcoTable [65] verifies each retrieved table against decomposed sub-questions with an LLM and joins the verified tables. Metadata-Reasoner [74] explicitly aims for a minimal yet sufficient set of tables, and retrieves the tables with an LLM agent that searches and calls profiling and joinability tools over many steps. ATR [36] adapts the number of tables to return by learning a score threshold for each question, and embeds joinable tables close to each other.

Although these methods account for join relationships in different ways, their retrieval and selection procedures do not necessarily ensure that the final set covers all attributes and value constraints while remaining connected and compact. In particular, a bridge table may be necessary to connect answer-relevant tables even when its contents do not closely match the question text. This is especially challenging in a data-lake setting, where many irrelevant tables appear to be joinable with relevant tables (Section 5.3). CORE-T and EcoTable evaluate a ranked list of data lake candidate tables either independently or against individual sub-questions, so they cannot discover a bridge table that is not retrieved as a candidate table or that no sub-question mentions. Mew instead measures how much of the answer-table sketch is still uncovered, and adds join paths of any length to cover the sketch. Its selection procedure thus can include bridge tables along join paths.

The number of tables to retrieve is another important distinction. Many methods use a fixed retrieval budget [2, 10, 45], a learned threshold [36], or an LLM to decide which tables are needed to answer the question [59, 65, 74]. In contrast, Mew does not require a predefined output size, and adapts the output size to the question. Mew selects tables using an explicit coverage criterion that adds at most $O(\rho \log m)$ times as many tables as needed. It stops when the grounded sketch is covered by the selected connected set, rather than when a relevance score crosses a threshold. Mew uses an LLM only to ground the sketch. We compare Mew with DMRAL, REAR, JAR, and CORE-T as representative methods for question decomposition, join-based candidate expansion, join-aware optimization, and LLM-based selection, respectively. We do not evaluate ATR because its learned threshold requires labeled training data that are unavailable for our target data lakes, or Metadata-Reasoner, whose code is not publicly available.

Text-to-SQL. Text-to-SQL methods [42, 53, 62] assume that the database is given. Given a database, schema linking selects the tables and columns relevant to a question. For example, FGTR [61] applies hierarchical LLM reasoning within a single database, selecting relevant columns and then relevant cells, and connects the selected tables by keys it estimates from the data. Schema linking and Mew address complementary stages of the task. Schema-linking methods identify relevant parts of a supplied database, whereas Mew searches a data lake to find a connected set of tables that can serve as input to a downstream text-to-SQL system. Question decomposition aim to simplify complex questions or the reasoning required to answer them, by decomposing complex questions and large input tables [71, 76], extracting or selecting a sub-table from an input table using a generated SQL [41, 48, 60], decomposing the reasoning process into step-by-step plans or table operations [35, 66, 70]. Multi-hop table question answering often

requires combining information across tables and associated text or passages [1, 11, 58, 73, 77]. They are related to Mew in their treatment of questions that require multiple pieces of evidence, but generally focus on reasoning over provided tables, passages, or retrieved inputs rather than on discovering a minimal connected set of tables in a data lake. Data lakes pose additional challenges beyond scale. Data lake tables can be heterogeneous, redundant, inconsistently described, or noisy, and their join relationships may not be declared in a common schema [26, 49]. Thus, text-to-SQL techniques do not by themselves solve the problem of discovering the necessary tables across a lake.

2.2 Table Discovery in Data Lakes

Table discovery in data lakes has been studied extensively in data management [18, 49]. Given a table as a query, joinable table search methods find tables that can join with the query via equi-joins [54, 78, 79] or semantic joins [15, 20, 25, 33, 37, 40, 47], and table union search methods find tables that can be unioned with the query [6, 19, 31, 32, 50, 55]. In addition, recent methods have integrated data lake tables [34] to create views [22] or to reclaim the query [17]. These methods generally take a table as the query and discover tables related to it through joins, unions, or integration. In contrast, Mew starts from a natural-language question and must identify the tables that jointly provide the information needed to answer it. Nevertheless, Mew leverages these table discovery techniques to identify join relationships in a data lake. Early work focused on keyword search over the metadata of data lake tables [7, 8, 27]. More recent systems consider natural-language questions over data lakes. Dense table retrieval [28] ranks tables by how similar their embedding is to the question embedding. Solo [64], Birdie [24], and Pneuma [4] retrieve a ranked list of tables that are each independently relevant to the question, which may miss tables that are needed only to connect relevant tables (i.e., bridge tables). Symphony [12] and Drama [30] retrieve data from a multi-modal lake or transform raw data from the web into tables to generate SQL queries, whereas Mew returns tables from an existing lake and leaves SQL query generation to any downstream model.

3 PRELIMINARIES

In this section, we define the join graph that connects the tables of a data lake and the answer-table sketch that represents a question. We then define the coverage of tables to a question, and use this to formalize the problem of minimal table retrieval (MTR) and show that it is NP-complete.

3.1 Join Graph

A data lake $\mathcal{T} = \{T_1, \ldots, T_n\}$ is a collection of tables, where each table $T \in \mathcal{T}$ has a set of attributes $\text{attr}(T)$. Unlike the traditional setting of text-to-SQL [43, 72], in a data lake setting [26, 49], we do not assume database identifiers, declared join keys, or column and table descriptions. Instead, tables are often pooled together from many sources, table and column names may be ambiguous and inconsistent, and schemas of unrelated tables may accidentally overlap [49]. Following work on joinable table discovery, columns A and B are *joinable* if they support an *equi-join*, which matches equal values [78, 79], or a *semantic join*, which matches values that refer to the same entities under different representations [15, 20, 25]. A and B are equi-joinable if the set containment of their distinct values,

$$\text{containment}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}, \tag{1}$$

is at least a threshold $\tau_v \in [0, 1]$, and a semantic join if the cosine similarity of their column embeddings is at least a threshold $\tau_c \in [0, 1]$. Both thresholds are constants fixed before retrieval (Section 4.1). Two tables are joinable if they have a pair of joinable columns.

We model the relationships among tables in $\mathcal{T}$ using a *join graph* $G_{\mathcal{T}} = (V, E, w)$, where

- $V = \mathcal{T}$,
- $E \subseteq V \times V$, where an undirected edge $(T_i, T_j) \in E$ indicates that T_i and T_j are joinable, and
- $w : E \to (0, 1]$ assigns each edge (T_i, T_j) the joinability score of the most joinable column pair of T_i and T_j, which combines their containment and embedding similarity (Section 4.1).

A set of tables $S \subseteq \mathcal{T}$ is *connected* if the induced subgraph $G_{\mathcal{T}}[S]$ is connected via edges in E, so that all tables in S can be combined by joins along these edges. Since $G_{\mathcal{T}}$ is inferred from the values and embeddings of data lake columns (Section 4.1), an edge indicates a likely join rather than a declared one.

3.2 Question Representation

Existing retrieval methods for table question-answering represent a question as natural-language sub-questions [45]. However, aligning natural-language questions to tabular data is challenging. Tabular data, especially those in data lakes, can have incomplete, ambiguous, or missing metadata. When semantics of table columns and values are ambiguous, it is difficult to align them with a natural-language question. Even if the question is decomposed into natural-language sub-questions, it is difficult to determine if sub-questions should be matched to table columns, rows, or values.

Instead, we propose to represent a natural-language question as the table that would answer it, or an *answer-table sketch*. Representing an information need as a table has been well-studied in data management. Query-by-Example lets users express queries as example tables [80], and table discovery methods take a query table as input [19, 32, 50]. Closest to our setting, CRUSH4SQL [39] prompts an LLM to hallucinate a schema for a question and uses the schema to retrieve columns. DCTR [38] decomposes a question into schema, value, and aggregation components and matches each independently against table and column names. Both representations serve only as retrieval queries. The hallucinated schema of CRUSH4SQL contains no values, and DCTR matches its value components against table and column names, not against the values stored in the columns. In contrast, each constraint in a sketch pairs a value with an attribute so Mew can check whether the column that matches the attribute contains the value. Formally, given a natural-language question Q, we construct a structured representation that captures its column and value semantics. The answer-table sketch of a question Q is

$$\mathcal{A}_Q = (C_Q, K_Q),$$

where C_Q is the set of required attributes and K_Q is the set of required constraints, such as filters on attribute values (Example 2).

C_Q contains the columns of the table that would answer Q, including the columns that the answer outputs and the columns that its constraints filter on, and each constraint in K_Q requires a column to contain a value or a range of values. An answer-table sketch describes the *evidence* that retrieved tables from a data lake must provide. The sketch is not a SQL query, in that it contains no join conditions or aggregates. In this section, we assume that $\mathcal{A}_Q$ is an oracle sketch that contains exactly the attributes and constraints that the tables answering Q must provide. Section 4.2 describes how Mew constructs an approximate $\mathcal{A}_Q$ from Q using an LLM, grounded in the data lake.

Example 2. Consider the question Q of Example 1, "*What is the percentage of carbon in double-bond compounds?*"

A sketch for Q has the attributes C_Q = {`bond_type`, `element`}, and two constraints, K_Q = {`bond_type = '='`, `element = 'c'`}. The data stores a double bond as `'='` in the column `bond_type` of the table `bond`, and carbon as `'c'` in the column `element` of the table `atom`. Section 4.2.2 shows how Mew approximates this sketch from Q and the data lake.

Each attribute and constraint of the sketch can be checked against a single table, so the sketch defines what a set of tables must cover, which we formalize next.

3.3 Coverage Model

Each table $T \in \mathcal{T}$ contributes partial information to answering question Q. The coverage of table T w.r.t. question Q is $\text{cov}(T, Q) \subseteq C_Q \cup K_Q$, which denotes the set of attributes and constraints in $\mathcal{A}_Q$ that can be satisfied using T. T covers an attribute $a \in C_Q$ if it contains a column that matches a, and covers a constraint if that column contains a value that satisfies the constraint. Section 4.2.1 describes how Mew decides whether a column matches an attribute and covers a constraint. For a set of tables $S \subseteq \mathcal{T}$, the coverage is

$$\text{Cov}(S, Q) = \bigcup_{T \in S} \text{cov}(T, Q).$$

Coverage is always measured against the current sketch $\mathcal{A}_Q$ of Q. The *coverage gap* of S is the set of sketch elements that S does not cover, $\text{Gap}(S, Q) = (C_Q \cup K_Q) \setminus \text{Cov}(S, Q)$, so S covers the sketch if and only if $\text{Gap}(S, Q) = \emptyset$. Since each table covers only part of the sketch, Mew uses the gap to decide which tables to add (Section 4.2.3).

The coverage function $f(S) = |\text{Cov}(S, Q)|$ has the following properties.

Lemma 1 (Monotonicity). *For any $S \subseteq S'$, $f(S) \leq f(S')$.*

Proof. Since $\text{Cov}(S, Q)$ is defined as a union of the coverage of each table $T \in S$ w.r.t. Q, adding tables to S cannot reduce the number of covered elements in $\mathcal{A}_Q$. Thus, $f(S)$ is monotone. □

Lemma 2 (Submodularity). *For any $S \subseteq S'$ and any table $T \notin S'$,*

$$f(S \cup \{T\}) - f(S) \geq f(S' \cup \{T\}) - f(S').$$

Proof. The marginal gain of adding a table T to S is $|\text{cov}(T, Q) \setminus \text{Cov}(S, Q)|$, the number of elements that T covers and S does not. Since $\text{Cov}(S, Q) \subseteq \text{Cov}(S', Q)$, every element that T newly covers for S' is also new for S, so the gain for S is at least the gain for S'. □

3.4 Problem Definition

Covering the sketch alone does not limit how many tables are retrieved, since any superset of a covering set also covers it. Retrieved tables are passed to a downstream text-to-SQL model, so every extra table consumes context and adds attributes that can be confused with the relevant ones. Irrelevant context is known to degrade LLM accuracy [57], and removing distractor tables improves downstream answer accuracy [2, 59]. Table 1 shows that adding distractor tables lowers downstream answer accuracy and inflates the prompt. We therefore seek the *minimal* set of tables that covers the question and can be joined.

Definition 1 (Minimal Table Retrieval (MTR)). *Given a question Q with answer-table sketch $\mathcal{A}_Q = (C_Q, K_Q)$, a data lake $\mathcal{T}$, and its join graph $G_{\mathcal{T}}$, find a set of tables $S \subseteq \mathcal{T}$ such that*

(*Coverage*)	$\text{Cov}(S, Q) = C_Q \cup K_Q$,
(*Connected*)	*S is connected,*
(*Minimized*)	*$\lvert S \rvert$ is minimized.*

We use sketch coverage and join-graph connectivity as proxies for answer sufficiency. While these conditions identify tables that collectively provide the required attributes and values and can potentially be joined, they do not guarantee that the resulting joins preserve the relationships needed to answer the question correctly. If no connected set covers all of $C_Q \cup K_Q$, e.g., because relevant tables lie in different components of $G_{\mathcal{T}}$, table selection covers as much of the sketch as the reachable candidates allow (Section 4.2.3).

To analyze complexity, we consider the decision version of MTR. Given an integer k, decide whether there exists a connected set $S \subseteq \mathcal{T}$ with $|S| \leq k$ and $\text{Cov}(S, Q) = C_Q \cup K_Q$. The minimization problem is at least as hard, since its optimal size answers the decision question for every k.

Theorem 3. *The decision version of MTR is NP-complete.*

Proof. We show that it is in NP and that it is NP-hard.

NP. Given a candidate set S, we can verify in polynomial time that ① $|S| \leq k$, ② $\text{Cov}(S, Q) = C_Q \cup K_Q$ by computing the union of the coverage of the tables in S, and ③ S is connected by a graph traversal of $G_{\mathcal{T}}[S]$.

NP-hardness. We reduce from the Minimum Set Cover problem. Let $U = \{e_1, \ldots, e_m\}$ be a universe and $\mathcal{X} = \{X_1, \ldots, X_n\}$ be a collection of subsets of U. The decision version of Minimum Set Cover asks whether there exist at most k sets in $\mathcal{X}$ whose union covers U. We construct an instance of our problem in polynomial time as follows.

- For each set $X_i \in \mathcal{X}$, create a table T_i with one column for each element of X_i.
- Let $C_Q \cup K_Q = U$, i.e., each element of the universe corresponds to a required attribute or constraint. Thus, each table T_i covers exactly the elements of X_i.
- $G_{\mathcal{T}}$ is constructed as a complete graph, so that all tables are connected.

Under this construction, selecting a set of tables S that covers all elements in U with $|S| \leq k$ is equivalent to selecting a set cover of size at most k. Thus, the problem is NP-hard. Since it is also in NP, it is NP-complete. □

Since MTR is NP-complete, we use a greedy approximation algorithm in Section 4.2.3.

4 MEW FRAMEWORK

Figure 2 shows the overall pipeline of Mew, which consists of an offline and online stage. In the *offline stage*, Mew profiles and indexes the data lake and builds its join graph $G_{\mathcal{T}}$ (Section 4.1). In the *online stage*, given a question Q, Mew selects a small connected set of tables that covers the answer-table sketch of Q (Section 4.2).

4.1 Offline Stage

The offline stage builds the resources that the online stage uses, namely indexes of the columns and tables for candidate discovery and the join graph for table selection. Following prior work on multi-table retrieval [45, 59], this stage is performed once per data lake and its resources can be reused for every question during the online stage. This cost can be parallelized across tables, and does not affect the online retrieval time.

Profiling and Indexing. Data lake tables often lack descriptions and have ambiguous, inconsistent column names [49]. For each column, Mew builds a value profile, including statistics (min, max, mean, standard deviation) and sample values for numerical columns, and all distinct non-null values for categorical columns. Mew also builds a semantic profile, in which an LLM describes each table and its columns from the schema and five sample rows. Mew embeds each column by its table name, column name, description, and distinct values into a column index, and each table by its name and description into a table index.

Join Graph. Since tables in a data lake often do not have join keys, Mew constructs $G_{\mathcal{T}}$ using equi-join and semantic join discovery methods, following the definition of joinability in Section 3.1. Mew computes the set containment (Eq. 1) between the distinct values of two columns, stored in their value profiles, and the cosine similarity between their embeddings in the column index. Mew computes a joinability score between two columns, $\beta \cdot$ cosine-similarity + $(1-\beta) \cdot$ containment, where $\beta \in [0, 1]$ balances semantic similarity against value overlap, and takes the highest-scoring column pair between two tables as the edge weight.

4.2 Online Stage

Given a question Q, the online stage of Mew constructs an answer-table sketch, retrieves and aligns candidate tables (Section 4.2.1), grounds the sketch in the candidates (Section 4.2.2), and selects a connected set of tables that covers the sketch (Section 4.2.3).

Section 3 assumes that an oracle sketch is given. In practice, such a sketch is not available. Instead, Mew approximates this oracle sketch by generating the sketch with an LLM and grounding it in sample rows of candidate tables (Sections 4.2.1 and 4.2.2).

4.2.1 Sketch Generation and Retrieval. To generate an answer-table sketch, we prompt an LLM with Q to describe an ideal table that answers Q, as a set of attributes and a set of constraints. For the question in Example 1, the LLM generates the attributes C_Q = {`Compound Type`, `Carbon Percentage`} and constraint K_Q = {`Compound Type = 'double-bond compounds'`}.

Candidate Discovery. Mew searches the column index and the table index with Q and the elements of $\mathcal{A}_Q$, and fuses the two rankings by reciprocal-rank fusion [14] into a fixed-size set $\mathcal{T}_Q \subseteq \mathcal{T}$ of candidate tables. The two indexes are complementary. The column index returns tables that contain one relevant column but may have ambiguous names, and the table index returns tables whose topic matches the question.

Algorithm 1: Candidate Alignment

Input: sketch $\mathcal{A}_Q = (C_Q, K_Q)$, candidates $\mathcal{T}_Q$, value profiles
Output: coverage $\mathrm{cov}(T, Q)$ of each candidate $T \in \mathcal{T}_Q$

```
foreach attribute a ∈ C_Q do
    foreach T ∈ T_Q do s_T(a) ← similarity of a and the column of
      T matched to a;
    ŝ(a) ← max_{T∈T_Q} s_T(a);
    add a to cov(T, Q) for each T with s_T(a) ≥ θ · ŝ(a)
foreach constraint on attribute a with value v in K_Q do
    foreach T ∈ T_Q do add the constraint to cov(T, Q) if the
      column of T matched to a contains v;
return cov(T, Q) for all T ∈ T_Q;
```

Candidate Alignment. A retrieval score indicates that a table is *related* to Q, but MTR requires knowing *which* sketch elements it covers (Section 3.3). Mew therefore aligns each candidate with the attributes and constraints in the sketch (Algorithm 1).

(i) Attributes to columns. Generic column names such as `id` or `name` are similar to many attributes. In addition to matching each column to the sketch attributes by the cosine similarity of their embeddings, Mew uses TF-IDF to deprioritize these generic column names. The similarity of a sketch attribute and a data lake column is $\beta \cdot$ cosine-similarity + $(1-\beta) \cdot$ TF-IDF-similarity, with the same weights as the joinability score (Section 4.1). The cosine similarity compares the embedding of the attribute to the embedding of the column in the column index, and the TF-IDF similarity compares the sketch attribute to the name and sample values of the data lake column. Mew matches the sketch attributes to the columns of each candidate T, so that each column matches at most one sketch attribute, and $s_T(a)$ is the similarity of attribute $a \in C_Q$ to its matched column. T covers a if $s_T(a) \geq \theta \cdot \hat{s}(a)$, where $\hat{s}(a)$ is the highest $s_{T'}(a)$ among the candidates and $\theta \leq 1$ is a tolerance, so that near-ties also cover a.

(ii) Constraints to values. A constraint on attribute a with value v is covered by T if the value profile of the column of T matched to a contains v, or, for a numeric column, if its range contains v.

The coverage $\mathrm{cov}(T, Q)$ consists of the attributes and constraints that T covers. Mew re-ranks the candidates by reciprocal-rank fusion of their retrieval rank and the fraction of the sketch that they cover, and passes the highest-ranked candidates to table selection.

4.2.2 Grounded Refinement. The initial sketch is generated without seeing the data lake, so its attributes and constraint values may not exist in any schema or column. Grounding the sketch requires showing the LLM data from the lake, but an entire lake is too large to fit into an LLM context and contains irrelevant tables that distract the model [57] (Table 1). Thus, Mew prompts the LLM with Q, the sketch, and only the highest-ranked candidates in $\mathcal{T}_Q$. Following CORE-T [59], Mew takes the schema and first rows of each candidate, truncating long cells. In a single call, the LLM (i) rewrites the sketch with column names and values that occur in the candidates, and (ii) returns a set F_Q of *focus tables* that likely hold the data that the answer needs.

Example 3. For the question of Example 1, the ten candidates shown to the LLM include the tables bond and atom, whose first rows are:

atom	
atom_id	element
TR000_1	cl
TR000_5	h
TR001_10	c

bond	
bond_id	bond_type
TR000_1_2	-
TR000_2_3	-
TR001_10_11	=

Since column bond.bond_type stores bond types as symbols and atom.element stores elements as lowercase chemical symbols, the LLM rewrites the sketch into $C_Q = \{\texttt{bond_type}, \texttt{element}\}$, $K_Q = \{\texttt{bond_type} = \texttt{'='}, \texttt{element} = \texttt{'c'}\}$, and returns bond and atom as focus tables. The LLM does not return the table connected, which covers no element of the sketch.

While the ranking relies on similarity scores, the LLM reads the rows in the context of Q to determine which tables hold the answer. However, the focus tables may not cover the sketch or be joinable. In particular, the LLM may omit *bridge tables*, as in Example 3, which cover no element of the sketch but are needed to join the tables that cover the sketch. Mew therefore uses the focus tables as *seeds* for table selection (Section 4.2.3) rather than as the answer.

Finally, Mew reruns candidate discovery and alignment (Section 4.2.1) with the refined sketch to retrieve tables that the initial sketch missed and to compute coverage of the refined elements. For example, with the refined sketch of Example 3, the table bond covers the attribute bond_type and the constraint bond_type = '=', and the table atom covers the attribute element and the constraint element = 'c'. Since F_Q is unordered, table selection uses the ranking from this rerun.

4.2.3 Table Selection. The aligned candidates are not guaranteed to *join*, nor to *cover* the sketch, and a bridge table needed to join them may not be a candidate at all. Since MTR is NP-complete (Theorem 3), we use an approximation algorithm. Mew selects tables with a greedy algorithm that exploits the monotonicity and submodularity of coverage (Lemmas 1 and 2).

Maximizing coverage with k tables. We first consider the relaxed problem $\max_{S \subseteq \mathcal{T}} f(S) = |\mathrm{Cov}(S, Q)|$ s.t. $|S| \le k$, which applies when the sketch cannot be fully covered.

Theorem 4. *The greedy algorithm that iteratively selects the table with the maximum marginal coverage gain covers at least a $(1 - 1/e)$ fraction of the elements covered by an optimal set of k tables.*

Proof. The function $f(S)$ is monotone and submodular (Lemmas 1 and 2), and $f(\emptyset) = 0$. At each step, the greedy algorithm selects the table that maximizes the marginal gain. By the result of Nemhauser et al. [51] for the greedy maximization of monotone submodular functions with at most k elements, after k iterations the greedy solution S_g satisfies $f(S_g) \ge (1 - 1/e)\, f(S^*)$, where S^* is an optimal set of k tables. □

Minimizing tables for full coverage. Consider the problem of covering all elements of the sketch while minimizing the number of tables, $\min_{S \subseteq \mathcal{T}} |S|$ s.t. $\mathrm{Cov}(S, Q) = C_Q \cup K_Q$.

Theorem 5. *The greedy algorithm that iteratively selects the table covering the largest number of uncovered elements covers the sketch with at most $O(\log m)$ times as many tables as a minimum cover, where $m = |C_Q \cup K_Q|$.*

Proof. This problem is an instance of the Set Cover problem, with universe $C_Q \cup K_Q$ and one set $\mathrm{cov}(T, Q)$ for each table T. The greedy algorithm selects at each step the table that covers the largest number of uncovered elements, which is the greedy algorithm for Set Cover. By its standard analysis [13], the solution size is at most $\ln m + 1$ times the optimum, so it is within a factor of $O(\log m)$. □

Algorithm 2: Join-Aware Table Selection

Input: candidates $\mathcal{T}_Q$ with coverage and ranking, join graph $G_{\mathcal{T}}$, focus tables F_Q
Output: a connected set of tables S

$S \leftarrow F_Q$ and the shortest join paths between them, or the highest-ranked candidate if $F_Q = \emptyset$;
while *true* **do**
  compute the shortest join path π_T from S to each $T \in \mathcal{T}_Q \setminus S$ in $G_{\mathcal{T}}$;
  foreach *reachable* T **do**
    $g_T \leftarrow |\mathrm{Gap}(S, Q)| - |\mathrm{Gap}(S \cup \pi_T \cup \{T\}, Q)|$;
  if $\max_T g_T = 0$ **then break**;
  $T^\star \leftarrow \arg\max_T g_T$, breaking ties by rank and path cost;
  $S \leftarrow S \cup \pi_{T^\star} \cup \{T^\star\}$;
return S;

Join-aware greedy selection. To also ensure connectivity, Mew grows a connected set S one *join path* at a time (Algorithm 2). The join graph determines *how* tables join, and the coverage from alignment determines *which* tables are needed. S starts from the focus tables F_Q (Section 4.2.2) and the shortest join paths between them, or from the highest-ranked candidate if $F_Q = \emptyset$. In each iteration, a multi-source shortest-path search from S finds the lowest-cost join path π_T to each candidate T, where an edge costs $1 - w$ (Section 4.1). Mew adds the path that minimizes the remaining gap $|\mathrm{Gap}(S \cup \pi_T \cup \{T\}, Q)|$, breaking ties by rank and path cost, and stops when no path reduces the gap.

Example 4. In Example 3, the focus tables bond and atom share no joinable column. Mew connects them through the shortest join path in $G_{\mathcal{T}}$, which adds the table connected, a bridge table that covers no sketch element and that the question does not mention. The gap is then empty, so Mew returns exactly these three tables.

Guarantee. Algorithm 2 extends the greedy algorithm of Theorem 5 with the connectivity condition.

Theorem 6. *Assume that for any candidate table T during greedy selection, the join path π_T connecting T to the current set S has at most ρ tables. Then, beyond its initial set, the join-aware greedy algorithm achieves an $O(\rho \log m)$-approximation for MTR.*

Proof. Let s^* be the size of an optimal solution S^*. Since S^* covers the current gap with s^* tables, some table of S^* covers at least a $1/s^*$ fraction of the gap, and by monotonicity (Lemma 1), its join path reduces the gap at least as much as the table alone. As in the analysis of greedy Set Cover (Theorem 5), the gap is therefore empty after at most $(\ln m + 1)\, s^*$ iterations. To ensure connectivity, each iteration adds a complete join path of at most ρ tables, so

Table 2: Statistics on BIRD, SPIDER, and the expanded benchmarks $\text{BIRD}_{\text{open-small}}$, $\text{BIRD}_{\text{open}}$, $\text{SPIDER}_{\text{open-small}}$, $\text{SPIDER}_{\text{open}}$.

Benchmark	# Tables	Avg. # Cols.	Total # Rows	Avg. # Distract.
BIRD	79	10.20	3.9M	21.50
$\text{BIRD}_{\text{open-small}}$	1,079	4.70	3.9M	92.41
$\text{BIRD}_{\text{open}}$	13,737	4.35	30.8M	1,061.70
SPIDER	81	5.44	540K	9.71
$\text{SPIDER}_{\text{open-small}}$	1,081	4.79	5.8M	55.56
$\text{SPIDER}_{\text{open}}$	16,650	4.73	100.3M	718.43

S stays connected. Thus, S contains at most $\rho\,(\ln m + 1)\,s^*$ tables beyond its initial set, leading to an $O(\rho \log m)$-approximation for MTR. □

The initial set S_0 connects the focus tables one at a time, each through one cheapest join path to the tables already selected. If each such path also has at most ρ tables, then $|S_0| \le (\rho+1)\max(|F_Q|, 1)$, so the $O(\rho \log m)$ bound also holds for the whole output when the LLM returns at most s^* focus tables. Although the bound grows with ρ, ρ is small in practice. For instance, gold queries in BIRD [43] and SPIDER [72] benchmarks join through at most two bridge tables, i.e., $\rho \le 3$.

Algorithm 2 addresses the three conditions of MTR (Definition 1) as follows.

- *Coverage.* It adds join paths until the gap is empty, or no reachable candidate reduces it.
- *Connectivity.* It connects the focus tables whenever a join path exists and adds only complete join paths.
- *Minimality.* Since finding a minimum set is NP-hard, Mew approximates it by adding only join paths that reduce the gap. Theorem 6 bounds the output size, and Section 5.2 compares it with the gold sets.

5 EXPERIMENTS

We evaluate Mew on six benchmarks built on BIRD [43] and SPIDER [72], whose data lakes range from 79 to over 16K tables. To create these benchmarks, we pool the tables of all databases of each benchmark, and add open data and web tables, following [45]. We show that Mew retrieves more accurate and smaller sets of tables than existing methods (Section 5.2), and that it remains the most accurate method as the number of joinable irrelevant tables, joins, and question difficulty increase (Section 5.3). We then measure its runtime and LLM cost (Section 5.4), and study the contribution of each step and the effect of the number of candidates, the embedding model, and the LLM (Section 5.5). Lastly, we show that the tables of Mew improve the execution accuracy of downstream text-to-SQL models (Section 5.6).

5.1 Setup

5.1.1 Environment. We implement Mew in Python using Sentence-Transformers, FAISS, and scikit-learn. By default if applicable, all methods use bge-base-en-v1.5 [68] as the embedding model and GPT-5.4-mini [52] with temperature 0 as the LLM. We vary both for Mew in Section 5.5. We set $\tau_v = 0.5$, $\tau_c = 0.7$ following REAR [2], $\theta = 0.9$, and $\beta = 0.6$. All experiments are run on a server with configurations similar to those of a p4d.24xlarge AWS EC2 machine with 8 A100 GPUs. The server has 2 AMD EPYC 7702 64-Core processors and 1TB RAM.

5.1.2 Benchmarks. We evaluate on six data lakes built from the dev splits of BIRD [43] and SPIDER [72] (see details in Table 2). Each data lake pools tables across all databases of a benchmark, without database identifiers or declared join keys, and uses the original questions and ground truth of the benchmark. BIRD and SPIDER contain only the tables of the original databases, with 1,524 questions over 79 tables and 1,034 questions over 81 tables.

To evaluate Mew on larger data lakes, we add open data and web tables from BirdWild and SpiderWild [45], keeping the questions and ground truth of BIRD and SPIDER. $\text{BIRD}_{\text{open-small}}$ and $\text{BIRD}_{\text{open}}$ add 1,000 and ∼13,600 random tables from BirdWild, and $\text{SPIDER}_{\text{open-small}}$ and $\text{SPIDER}_{\text{open}}$ add 1,000 and ∼16,500 random tables from SpiderWild. Since Mew only considers join relationships, we merge tables with identical schemas into one table and omit empty tables.

To measure the noise in each lake, a main characteristic of data lakes [49], we count the number of *distractors*, which are tables that are not needed to answer a question but can be mistaken as relevant. As a proxy, we count the tables that share a column name with a gold table, excluding the gold tables of the question, since without declared join keys, such tables can appear joinable with the gold table. We average this count over the gold tables of each question, and Table 2 reports the mean over all questions. Gold tables have about 49 and 74 times as many distractors in $\text{BIRD}_{\text{open}}$ and $\text{SPIDER}_{\text{open}}$ as in BIRD and SPIDER, respectively, and we show how these distractors affect retrieval accuracy in Section 5.3 .

5.1.3 Baselines. We compare Mew with four existing multi-table retrieval methods.

• **DMRAL** [45] retrieves multiple tables for questions in a data lake setting. It decomposes each question into natural-language sub-questions, retrieves tables for each sub-question, and combines them into groups of joinable tables.[1]

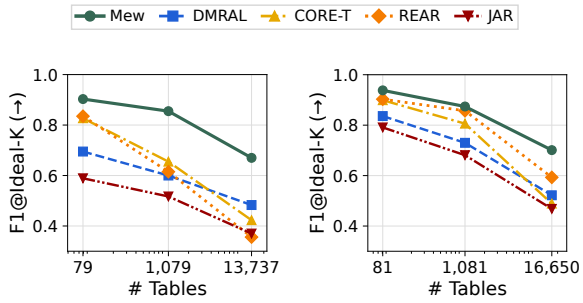
• **CORE-T** [59] uses an LLM to select a set of tables that can be joined to answer the question.[2]

• **REAR** [2] retrieves tables for the question, expands the table set with tables whose columns are joinable, and re-ranks the tables with a cross-encoder.[3]

• **JAR** [10] formulates join-aware table retrieval as a mixed-integer program that selects tables by their relevance to the question and their pairwise join compatibility. We use its Contriever-based variant with $K = 5$, and set a limit of 600 seconds per question, after which JAR returns the best solution found so far.[4]

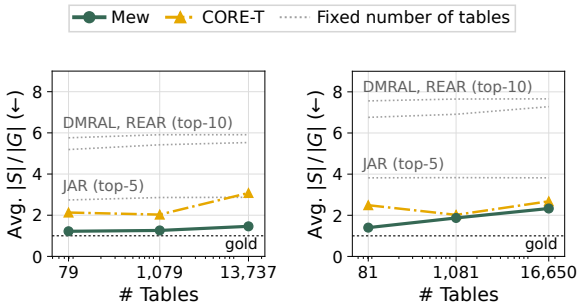
We set DMRAL and REAR to return ten tables, and JAR to return five, since its mixed-integer program does not finish for ten. We set these numbers to be at least the size of the largest gold set, which has five tables in BIRD and four in SPIDER, so that every method can return all gold tables of every question.

[1] https://github.com/DataAutonomyLab/DMRAL
[2] https://github.com/UKPLab/emnlp2026-core-t
[3] https://github.com/CoRAL-ASU/REaR
[4] https://github.com/peterbaile/jar



(a) F1@Ideal-K on BIRD lakes (b) F1@Ideal-K on SPIDER lakes

Figure 3: F1@Ideal-K of all methods on BIRD as we scale up to 13K tables in $\text{BIRD}_{\text{OPEN}}$ (Figure a), and on SPIDER as we scale the data lake to 16K in $\text{SPIDER}_{\text{OPEN}}$ (Figure b).



(a) $|S|/|G|$ on BIRD lakes (b) $|S|/|G|$ on SPIDER lakes

Figure 4: Average ratio $|S|/|G|$ of the number of returned tables ($|S|$) to the number of gold tables $|G|$ on the BIRD data lakes (a) and the SPIDER data lakes (b). The dashed line marks the size of the gold set. DMRAL and REAR return ten tables and JAR returns five (Section 5.1.3), so their ratios are shown as reference lines.

5.1.4 Metrics. We call the tables needed to answer a question, as given by the ground truth of the benchmark, its *gold set* G. For a question with gold set of tables G, the precision, recall, and F1 of a set of tables S are $P = |S \cap G|/|S|$, $R = |S \cap G|/|G|$, and $F1 = 2PR/(P + R)$, averaged over all questions. Since DMRAL, REAR, and JAR return a fixed number of tables and cannot know $|G|$ in advance, we compare the methods at Ideal-K, the size of the gold set [3, 46]. P@Ideal-K, R@Ideal-K, and F1@Ideal-K evaluate the $|G|$ highest-ranked tables of each method, and EM@Ideal-K is the fraction of questions whose $|G|$ highest-ranked tables are exactly G. To measure minimality, we evaluate the set S that each method returns by the average ratio $|S|/|G|$ and the fraction of questions where $S = G$. To measure downstream NL2SQL accuracy, we pass the returned tables to a text-to-SQL model and report the fraction of questions whose generated SQL query returns the same result as the gold query (execution accuracy). We also report the error rate, which is the fraction of generated SQL queries that fail to execute. To evaluate efficiency, we report the average runtime per question and the average number of LLM tokens per question. All scores are averaged over all questions and over five runs.

5.2 Effectiveness

Retrieval accuracy. Table 3 reports the P@Ideal-K, R@Ideal-K, and EM@Ideal-K of all methods. Mew achieves the highest scores on all six data lakes. On $\text{BIRD}_{\text{OPEN}}$, Mew outperforms the baselines by 50.9–104.8% in P@Ideal-K, 32.9–80.3% in R@Ideal-K, and 71.7–143.3% in EM@Ideal-K. On $\text{SPIDER}_{\text{OPEN}}$, Mew outperforms the baselines by 19.9–51.9% in P@Ideal-K, 17.4–48.7% in R@Ideal-K, and 21.6–71.1% in EM@Ideal-K.

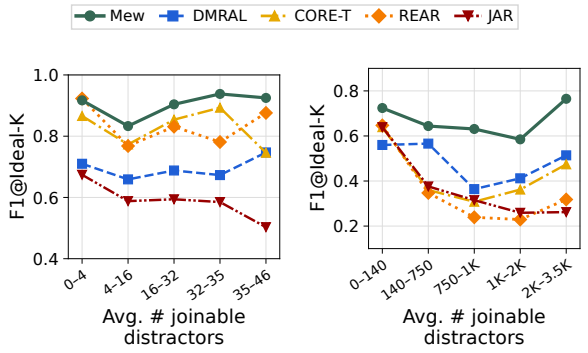
As shown in Figure 3, Mew consistently outperforms the baselines in F1@Ideal-K as the data lake scales up to 13K tables in $\text{BIRD}_{\text{OPEN}}$ (Figure 3a) and to 16K tables in $\text{SPIDER}_{\text{OPEN}}$ (Figure 3b). As the data lakes grow, the F1@Ideal-K of all methods decreases, since the noisy tables added to the data lakes distract the retrieval methods. The F1@Ideal-K of Mew decreases the least, by 25.8% from BIRD to $\text{BIRD}_{\text{OPEN}}$ and by 25.3% from SPIDER to $\text{SPIDER}_{\text{OPEN}}$, compared with 30.5–57.4% and 34.3–45.6% for the baselines. As a result, the margin of Mew over the baselines grows with the size of the data lake, from 8.1–53.3% on BIRD to 38.7–88.2% on $\text{BIRD}_{\text{OPEN}}$, and from 3.9–18.6% on SPIDER to 18.2–49.8% on $\text{SPIDER}_{\text{OPEN}}$.

DMRAL retrieves tables for each sub-question, so a table that matches only part of the question can rank above a gold table that matches all of it. This hurts DMRAL on the two smaller SPIDER data lakes, where most questions need a single table, but helps on $\text{BIRD}_{\text{OPEN}}$, where a narrow sub-question still finds its gold table among the many distractors that match the question as a whole, so DMRAL is the strongest baseline there. CORE-T uses an LLM to select tables only among the top candidates of dense retrieval, so gold tables that rank below distractors never reach the LLM. REAR expands the retrieved tables with joinable tables, so it is the strongest baseline on the SPIDER data lakes but drops sharply on $\text{BIRD}_{\text{OPEN}}$, where gold tables have many joinable distractors (Section 5.3). JAR selects a fixed number of tables that maximize relevance and pairwise join compatibility, which can favor joinable but irrelevant tables. In contrast, Mew retrieves candidates with the sketch instead of the question, and grounds the sketch using data lake tables (Section 4.2.2). From BIRD to $\text{BIRD}_{\text{OPEN}}$, the recall of the returned set of CORE-T drops by 52.8%, while that of Mew drops by 24.7%. Thus, the sketch makes the retrieval of Mew less sensitive to noisy tables in larger data lakes.

Minimality. Figure 4 shows the average ratio $|S|/|G|$ as the data lakes grow. Mew returns 1.22–1.46 times as many tables as the gold set on the BIRD data lakes (Figure 4a), and this ratio remains consistent as the data lake grows by two orders of magnitude. On the SPIDER data lakes (Figure 4b), Mew returns 1.40–2.33 times as many tables as the gold set. CORE-T, the only baseline that returns a dynamic number of tables, returns 2.13–3.08 times as many tables as the gold set on the BIRD data lakes and 2.02–2.68 times on the SPIDER data lakes, since it asks the LLM for a joinable subset without a stopping criterion and then adds candidates that are joinable with the selected tables. Mew instead adds a table only when it covers an attribute or constraint of the sketch that the previously-selected tables do not cover, or when it lies on a join path between such tables, and it stops when no join path reduces

Table 3: P@Ideal-K (P), R@Ideal-K (R), and EM@Ideal-K (EM) of Mew and the baselines on all data lakes.

Method	BIRD			$BIRD_{OPEN-SMALL}$			$BIRD_{OPEN}$			SPIDER			$SPIDER_{OPEN-SMALL}$			$SPIDER_{OPEN}$		
	P	R	EM	P	R	EM	P	R	EM	P	R	EM	P	R	EM	P	R	EM
DMRAL	0.695	0.695	0.465	0.600	0.600	0.354	0.483	0.483	0.265	0.836	0.836	0.729	0.730	0.730	0.609	0.522	0.522	0.406
CORE-T	0.830	0.825	0.692	0.680	0.643	0.471	0.479	0.396	0.256	0.899	0.899	0.853	0.817	0.800	0.703	0.500	0.484	0.375
REAR	0.835	0.835	0.684	0.615	0.615	0.418	0.356	0.356	0.219	0.903	0.903	0.834	0.857	0.857	0.770	0.593	0.593	0.491
JAR	0.589	0.589	0.385	0.517	0.517	0.295	0.370	0.370	0.187	0.791	0.791	0.698	0.681	0.681	0.548	0.468	0.468	0.349
Mew	**0.918**	**0.895**	**0.795**	**0.884**	**0.840**	**0.710**	**0.729**	**0.642**	**0.455**	**0.941**	**0.936**	**0.887**	**0.882**	**0.869**	**0.790**	**0.711**	**0.696**	**0.597**



(a) BIRD (b) $BIRD_{OPEN}$

Figure 5: F1@Ideal-K on BIRD and $BIRD_{OPEN}$ for five groups of questions of equal size, ordered by the average number of joinable distractors of their gold tables.

Table 4: F1@Ideal-K on $BIRD_{OPEN}$ and $SPIDER_{OPEN}$ by question difficulty (simple, moderate, challenging).

Method	$BIRD_{OPEN}$			$SPIDER_{OPEN}$		
	Simple	Moderate	Chall.	Simple	Moderate	Chall.
DMRAL	0.500	0.453	0.474	0.526	0.576	0.492
CORE-T	0.468	0.352	0.419	0.443	0.543	0.472
REAR	0.383	0.306	0.347	0.569	0.655	0.560
JAR	0.376	0.350	0.399	0.440	0.496	0.487
Mew	**0.701**	**0.628**	**0.596**	**0.699**	**0.749**	**0.646**

the gap (Section 4.2.3). DMRAL, REAR, and JAR return about the same number of tables for every question (top-10 for DMRAL and REAR and top-5 for JAR, Section 5.1.3), so their ratios stay between 2.7 and 7.7. The small sets of retrieved tables of Mew are also correct. Mew returns exactly the gold set for 32.5–61.5% of the questions on the BIRD data lakes and 27.9–66.3% on the SPIDER data lakes, compared with 17.3–21.5% and 10.8–21.6% for CORE-T, while DMRAL, REAR, and JAR never return exactly the gold set. Thus, Mew returns sets with sizes close to the size of the gold sets, without being given a fixed number of tables to return.

5.3 Robustness

Joinable distractors. We study how joinable distractors (Section 5.1.2) affect retrieval accuracy on BIRD and $BIRD_{OPEN}$, using the average number of joinable distractors of the gold tables of each question. We evenly split the questions into five groups based on their number of average joinable distractors.

As shown in Figure 5, Mew achieves the highest F1@Ideal-K, remaining robust to up to 3.5K joinable distractors in $BIRD_{OPEN}$. JAR and REAR are the most sensitive to joinable distractors. JAR maximizes the pairwise join compatibility of the selected tables, so a set with joinable distractors can score higher than the gold set. REAR expands the retrieved tables with joinable tables before re-ranking them, so it is accurate on BIRD, where gold tables have few joinable distractors, but not on $BIRD_{OPEN}$. Similarly, CORE-T expands the tables selected by its LLM with joinable candidates, which is consistent with its decrease in the last group on BIRD. DMRAL uses joinability only to group the retrieved tables, but it matches each sub-question to columns, so distractors that share columns with a gold table also match its sub-question. In contrast, Mew selects tables that cover attributes and constraints of the sketch, and traverses the join graph only to connect them, adding the bridge tables on the join paths between them (Section 4.2.3). Thus, Mew selects a joinable distractor only when it covers the sketch or the answer requires a join through it. On $BIRD_{OPEN}$, Mew, DMRAL, CORE-T, and REAR improve in the last group, which suggests that F1@Ideal-K also depends on other properties of the questions in each group, such as the number of gold tables.

Number of joins. Mew achieves the highest F1@Ideal-K on $BIRD_{OPEN}$ and $SPIDER_{OPEN}$ for every number of joins in the gold query. Mew outperforms the best baseline, DMRAL, on $BIRD_{OPEN}$ by 26.2–42.3%, and REAR on $SPIDER_{OPEN}$ by 15.2–31.2%, on questions with 0–3+ joins. Like CORE-T and REAR, Mew's accuracy decreases as the number of expected joins increases, since a question that expects more joins also has bridge tables that match no part of the question. REAR expands the retrieved tables with joinable tables, so with more retrieved tables, its expanded set also includes more joinable distractors. CORE-T shows the LLM only the top candidates of dense retrieval, which may not include bridge tables, since they rank low for the question. In contrast, DMRAL decomposes the question into sub-questions and retrieves tables for each sub-question, so it is less sensitive to more expected joins. JAR jointly maximizes the relevance of the selected tables to the question and their pairwise join compatibility, which favors connected sets of several tables, such as the gold sets of questions with more joins. However, the F1@Ideal-K of Mew is up to 121.1% higher than that of JAR and up to 79.7% higher than DMRAL. Thus, Mew remains the most accurate as questions require more joins, since it selects tables that cover the attributes and values of the sketch and connects them through join paths, including bridge tables that match no attribute of the sketch (Section 4.2.3).

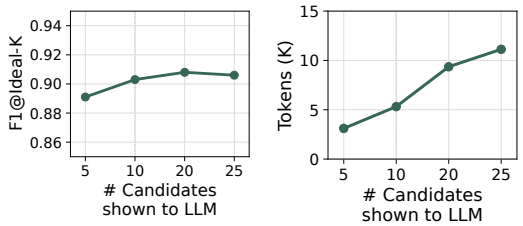
Table 5: Average runtime (seconds) and LLM tokens (thousands) per question. REAR and JAR do not use an LLM.

	DMRAL	CORE-T	REAR	JAR	MEW
Runtime (s)					
BIRD	2.30	3.70	10.28	155.89	2.28
$\text{BIRD}_{\text{OPEN}}$	2.32	3.28	5.05	340.05	2.68
SPIDER	2.09	4.08	2.07	42.75	2.19
$\text{SPIDER}_{\text{OPEN}}$	1.95	4.32	4.36	118.77	3.51
Tokens (K)					
BIRD	1.60	7.73	–	–	5.32
$\text{BIRD}_{\text{OPEN}}$	1.54	3.00	–	–	2.96
SPIDER	1.46	5.07	–	–	2.25
$\text{SPIDER}_{\text{OPEN}}$	1.53	5.42	–	–	2.37

Question difficulty. Table 4 reports F1@Ideal-K by the difficulty levels of questions in $\text{BIRD}_{\text{OPEN}}$ and $\text{SPIDER}_{\text{OPEN}}$. We use the difficulty levels annotated in the original benchmarks [43, 72], which depend on the number of SQL components, selections, and conditions in the gold query. MEW outperforms the best baseline, DMRAL, by 25.7% on $\text{BIRD}_{\text{OPEN}}$ and REAR by 15.4% on $\text{SPIDER}_{\text{OPEN}}$ for challenging questions. The accuracy of MEW decreases as questions become more difficult. A challenging question has more conditions, literals, and aggregations, so the sketch has more attributes and constraints. Since table selection covers every element of the sketch, an element that is not needed or is aligned to a wrong column can add a wrong table, and an element that is missing can leave out a gold table. The accuracy of the baselines varies less with difficulty, since it is limited by distractors at every level. CORE-T, REAR, and JAR rank tables by their similarity to the question, which does not depend on the complexity of the gold query, and DMRAL splits a complex question into simpler sub-questions. MEW remains the most accurate at every difficulty level, since grounded refinement aligns the sketch with the columns of the candidates (Section 4.2.2), and table selection covers every element of the sketch (Section 4.2.3).

5.4 Efficiency

Table 5 reports the average runtime and the average number of LLM tokens (input and output) per question on BIRD, $\text{BIRD}_{\text{OPEN}}$, SPIDER, and $\text{SPIDER}_{\text{OPEN}}$. Most of the runtime of MEW is spent on its two LLM calls, one to generate the sketch (Section 4.2.1) and one for grounded refinement (Section 4.2.2). Similar to MEW, CORE-T prompts an LLM with a fixed number of candidates with sample rows to select tables. CORE-T also prompts the LLM with the joinability of each pair of candidate tables and asks the LLM to return every group of joinable candidates, while MEW asks the LLM only to refine the sketch and name the focus tables. Thus, MEW is 1.2–1.9× faster than CORE-T and uses up to 2.1× fewer input tokens and up to 10.4× fewer output tokens. JAR is 20–127× slower than MEW, since it solves a mixed-integer program for each question. MEW is comparable to or faster than REAR, which uses no LLM, and comparable to DMRAL, which prompts an LLM only with the names and columns of tables and thus uses the fewest tokens. However, MEW outperforms REAR by 3.9–88.2% and DMRAL by 12.2–38.7% in F1@Ideal-K (Table 3). CORE-T, JAR, and DMRAL select tables only among a fixed number of candidates, and REAR adds only tables that are directly joinable with its retrieved tables. MEW instead searches join paths of any length in the whole join graph to find bridge tables that are not among its candidates. Its table selection takes less than 0.01 seconds on BIRD and SPIDER, but occupies 13% and 34% of its runtime on $\text{BIRD}_{\text{OPEN}}$ and $\text{SPIDER}_{\text{OPEN}}$. Thus, MEW achieves the highest accuracy at a runtime comparable to that of the LLM-based baselines and with fewer tokens than CORE-T.



(a) F1@Ideal-K **(b) Average token cost**

Figure 6: F1@Ideal-K and average LLM tokens per question of MEW on BIRD as we vary the number of candidates shown to the LLM in grounded refinement.

Table 6: Ablation of the steps of MEW on BIRD. Each row adds one step to the row above. P, R, F1 are at Ideal-K

Variant	P	R	F1
Candidate retrieval with Q	0.745	0.745	0.745
+ Candidate alignment to $\mathcal{A}_Q$	0.768	0.768	0.768
+ Table selection	0.835	0.746	0.776
+ Grounded refinement (w/o focus)	0.897	0.786	0.823
+ Table Selection (w/ focus) – **MEW**	**0.918**	**0.895**	**0.903**

Table 7: P@Ideal-K, R@Ideal-K, and F1@Ideal-K of MEW on BIRD with different numbers of candidates $|\mathcal{T}_Q|$, embedding models, and LLMs. * marks the default setting.

	Method	P	R	F1
Candidate Pool Size	$\|\mathcal{T}_Q\| = 10$	0.919	0.896	0.904
	$\|\mathcal{T}_Q\| = 25$*	0.918	0.895	0.903
	$\|\mathcal{T}_Q\| = 50$	0.919	0.897	0.905
Embedding Model	bge-base-en-v1.5 [68]*	0.918	0.895	0.903
	UAE-Large-V1 [44]	0.913	0.891	0.899
	EmbeddingGemma [56]	0.909	0.885	0.893
LLM	GPT-5.4-mini [52]*	0.918	0.895	0.903
	Qwen2.5-7B-Instruct [69]	0.888	0.802	0.831
	Llama-3.1-8B-Instruct [23]	0.823	0.792	0.803

5.5 Ablation Studies

Steps of MEW. Table 6 adds the steps of MEW one at a time on BIRD. Retrieving candidates with the natural-language question Q instead of the sketch achieves an F1@Ideal-K of 0.745. Aligning the candidates with the attributes of the initial sketch $\mathcal{A}_Q$ (Section 4.2.1) improves F1@Ideal-K by 3.1%. Selecting tables from the

candidates with the initial sketch (Section 4.2.3) improves P@Ideal-K by 8.7%, since it only keeps candidates that cover an attribute or constraint of the sketch. However, R@Ideal-K decreases since the initial sketch can miss attributes that are named differently in the data lake. Sketch refinement (Section 4.2.2) grounds the sketch with the attributes and values of data lake candidates and improves F1@Ideal-K by 6.1%. Starting table selection from the focus tables improves R@Ideal-K by 13.9% and F1@Ideal-K by 9.7%, since the LLM chooses tables that contain columns and rows that are relevant to the sketch and question. Thus, every step contributes, and the two steps that ground the sketch in the candidate tables, sketch refinement and the focus tables, improve F1@Ideal-K the most, by 16.4% together.

Varying number of candidates. Figure 6 shows the F1@Ideal-K and the average LLM token cost of Mew on BIRD as we vary the number of candidates shown to the LLM during grounded sketch refinement (Section 4.2.2). F1@Ideal-K improves only slightly with more candidates, but the token cost grows linearly with the number of candidates, by 2.1× from 10 to 25 candidates. Thus, ten candidates give the LLM enough context to ground the sketch, and we use ten candidates in all experiments to keep the token cost low. Similarly, varying the number of retrieved candidates $|\mathcal{T}_Q|$ from 10 to 50 does not have much effect on F1@Ideal-K (Table 7), since the LLM sees only the ten highest-ranked candidates regardless of $|\mathcal{T}_Q|$ (Section 4.2.2), and table selection adds a candidate only when it reduces the gap (Section 4.2.3).

Varying embedding models. Mew uses the embedding model to build the column and table indexes and to measure semantic joinability in the join graph in the offline phase (Section 4.1), and to align the sketch attributes with the columns of the candidates at query time (Section 4.2.1). Replacing bge-base-en-v1.5 [68] with UAE-Large-V1 [44] or EmbeddingGemma-300m [56] reduces F1@Ideal-K by at most 1.1% (Table 7). Thus, Mew does not depend on a particular embedding model, since the sketch and grounded refinement, not the embeddings alone, decide which tables are selected.

Varying LLMs. Mew uses the LLM to generate the initial sketch (Section 4.2.1) and for grounded refinement (Section 4.2.2). Replacing GPT-5.4-mini with the open models Qwen2.5-7B-Instruct and Llama-3.1-8B-Instruct [23, 69] reduces F1@Ideal-K by 8.0% and 11.1%. With Qwen2.5-7B-Instruct, Mew is on par with CORE-T using GPT-5.4-mini (0.831 vs. 0.827). Thus, the LLM has the largest effect among the components we vary, since a stronger LLM grounds the sketch more accurately. Mew remains competitive even with a smaller open model.

5.6 Downstream Accuracy

Execution accuracy. We evaluate how the tables returned by each method affect downstream text-to-SQL. Following related work [2, 21, 59], we prompt a text-to-SQL LLM with the question and the returned tables, using the same prompt for all methods. We use LLMs of varying sizes, Llama-3.2-3B-Instruct, Llama-3.1-8B-Instruct [23], and GPT-5.4-mini [52], and we report the execution accuracy of the generated queries. As shown in Table 8, Mew achieves the highest execution accuracy with all three LLMs, outperforming the best baseline, CORE-T, by 56.5% and 96.4% with the two Llama models, and DMRAL by 13.7% with GPT-5.4-mini. The LLM needs every gold table to write a correct query, and every extra table adds columns and join paths that it can choose by mistake. DMRAL returns most gold tables but also many distractors, and CORE-T, REAR, and JAR miss more gold tables, while Mew returns exactly the gold set for 32.5% of the questions on $\text{BIRD}_{\text{OPEN}}$, compared with 17.3% for CORE-T (Section 5.2). The margin of Mew is largest with the smaller open LLMs, which are more sensitive to irrelevant tables. Thus, the minimal sets of Mew lead to more accurate SQL queries, especially with smaller text-to-SQL LLMs.

Table 8: Execution accuracy (EX) and error rate (Err.) of text-to-SQL LLMs of three sizes, given the tables returned by each method on $\text{BIRD}_{\text{OPEN}}$.

Method	Llama-3.2-3B		Llama-3.1-8B		GPT-5.4-mini	
	EX (↑)	Err. (↓)	EX (↑)	Err. (↓)	EX (↑)	Err. (↓)
DMRAL	0.096	0.576	0.110	0.288	0.365	0.063
CORE-T	0.108	0.536	0.138	0.352	0.209	0.085
REAR	0.083	0.699	0.081	0.433	0.216	0.092
JAR	0.073	0.616	0.109	0.461	0.141	0.329
Mew	**0.169**	**0.410**	**0.271**	**0.223**	**0.415**	**0.043**

Error rate. With all three LLMs, the tables of Mew also lead to the fewest queries that fail to execute, 23.5% fewer than those of CORE-T with Llama-3.2-3B-Instruct, and 22.6% and 31.7% fewer than those of DMRAL with Llama-3.1-8B-Instruct and GPT-5.4-mini. Most failed queries reference a table or column that is missing because a gold table was not returned, or a column name that is ambiguous because distractors share it with a gold table. The fixed-size sets of DMRAL, REAR, and JAR lead to the highest error rates with Llama-3.2-3B-Instruct, the smallest LLM, which is the most sensitive to distractors. Thus, the minimal and connected sets of Mew lead to SQL queries that execute more often.

6 CONCLUSION

We formalize the problem of minimal table retrieval (MTR), which aims to retrieve tables from a data lake to answer a natural-language question, where the retrieved tables must cover the question, be connected, and be minimal. We show that this problem is NP-complete. To solve this problem, we present Mew, which represents a question as an answer-table sketch of the attributes and value constraints that the answer requires, and grounds the sketch in sample rows of candidate tables so that its elements match the columns and values of the data lake. A join-aware greedy algorithm then selects tables that cover the sketch and the bridge tables that connect them, with an $O(\rho \log m)$-approximation guarantee. Our experiments on six benchmarks built from BIRD and SPIDER, with up to 16K tables and 100M rows, show that Mew retrieves more accurate and smaller sets of tables than existing methods, remains the most accurate as data lakes grow and questions require more joins, and improves the execution accuracy of downstream text-to-SQL models. In future work, we plan to consider semantic unions [19, 32], value-joinability [20], and multi-attribute joins [16]. We also plan to extend answer-table sketches to questions whose answers combine tables with text or other modalities, and to use feedback from the downstream text-to-SQL model to refine the sketch. The join paths that Mew returns could also be given to the text-to-SQL model as additional evidence to verify the joins in the generated SQL queries.

REFERENCES

[1] Ankush Agarwal, Chaitanya Devaguptapu, and Ganesh S. 2025. Hybrid Graphs for Table-and-Text based Question Answering using LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 858–875.

[2] Rishita Agarwal, Himanshu Singhal, Peter Baile Chen, Manan Roy Choudhury, Dan Roth, and Vivek Gupta. 2026. REaR: Retrieve, Expand and Refine for Effective Multitable Retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 39360–39374.

[3] Javed A Aslam and Emine Yilmaz. 2005. A geometric interpretation and analysis of r-precision. In *Proceedings of the 14th ACM international conference on Information and knowledge management*. 664–671.

[4] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[5] Gaurav Bhalotia, Arvind Hulgeri, Charuta Nakhe, Soumen Chakrabarti, and S. Sudarshan. 2002. Keyword Searching and Browsing in Databases using BANKS. In *ICDE*. 431–440.

[6] Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, and Nikolaos Konstantinou. 2020. Dataset Discovery in Data Lakes. In *ICDE*. 709–720.

[7] Dan Brickley, Matthew Burgess, and Natasha F. Noy. 2019. Google Dataset Search: Building a search engine for datasets in an open Web ecosystem. In *WWW*. 1365–1375.

[8] Michael J. Cafarella, Alon Y. Halevy, and Nodira Khoussainova. 2009. Data Integration for the Relational Web. *Proc. VLDB Endow.* 2, 1 (2009), 1090–1101.

[9] Peter Baile Chen, Yi Zhang, Mike Cafarella, and Dan Roth. 2025. Can we retrieve everything all at once? ARM: An Alignment-Oriented LLM-based Retrieval Method. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 30298–30317.

[10] Peter Baile Chen, Yi Zhang, and Dan Roth. 2024. Is table retrieval a solved problem? exploring join-aware multi-table retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (volume 1: long papers)*. 2687–2699.

[11] Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1026–1036.

[12] Zui Chen, Zihui Gu, Lei Cao, Ju Fan, Sam Madden, and Nan Tang. 2023. Symphony: Towards Natural Language Query Answering over Multi-modal Data Lakes. In *CIDR*. 1–7.

[13] V. Chvátal. 1979. A Greedy Heuristic for the Set-Covering Problem. *Mathematics of Operations Research* 4, 3 (1979), 233–235.

[14] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *SIGIR*. 758–759.

[15] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.* 16, 10 (2023), 2458–2470.

[16] Mahdi Esmailoghli, Jorge-Arnulfo Quiané-Ruiz, and Ziawasch Abedjan. 2022. MATE: Multi-Attribute Table Extraction. *Proc. VLDB Endow.* 15, 8 (2022), 1684–1696.

[17] Grace Fan, Roee Shraga, and Renée J. Miller. 2024. Gen-T: Table Reclamation on Data Lakes. In *ICDE*. 3532–3545.

[18] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *SIGMOD Conference Companion*. 69–75.

[19] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *Proc. VLDB Endow.* 16, 7 (2023), 1726–1739.

[20] Grace Fan, Eden Wu, Majid Daliri, and Juliana Freire. 2026. MosaicJoin: Compact Semantic Sketches for Value-Level Join Discovery. *Proc. VLDB Endow.* 19, 11 (2026), 3745–3758.

[21] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (2024), 1132–1145.

[22] Yue Gong, Zhiru Zhu, Sainyam Galhotra, and Raul Castro Fernandez. 2023. Ver: View Discovery in the Wild. In *ICDE*. 503–516.

[23] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[24] Yuxiang Guo, Zhonghao Hu, Yuren Mao, Baihua Zheng, Yunjun Gao, and Mingwei Zhou. 2025. Birdie: Natural Language-Driven Table Discovery Using Differentiable Search Index. *Proc. VLDB Endow.* 18, 7 (2025), 2070–2083.

[25] Yuxiang Guo, Yuren Mao, Zhonghao Hu, Lu Chen, and Yunjun Gao. 2025. Snoopy: Effective and Efficient Semantic Join Discovery via Proxy Columns. *IEEE Trans. Knowl. Data Eng.* 37, 5 (2025), 2971–2985.

[26] Rihan Hai, Christos Koutras, Christoph Quix, and Matthias Jarke. 2023. Data Lakes: A Survey of Functions and Systems. *IEEE Trans. Knowl. Data Eng.* 35, 12 (2023), 12571–12590.

[27] Alon Y. Halevy, Flip Korn, Natalya Fridman Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. 2016. Goods: Organizing Google's Datasets. In *SIGMOD*. 795–806.

[28] Jonathan Herzig, Thomas Müller, Syrine Krichene, and Julian Eisenschlos. 2021. Open domain question answering over tables via dense retrieval. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 512–519.

[29] Vagelis Hristidis and Yannis Papakonstantinou. 2002. DISCOVER: Keyword Search in Relational Databases. In *Proc. VLDB Endow.* 670–681.

[30] Chuxuan Hu, Maxwell Yang, James Weiland, Yeji Lim, Suhas Palawala, and Daniel Kang. 2025. Drama: Unifying Data Retrieval and Analysis for Open-Domain Analytic Queries. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–28.

[31] Xuming Hu, Shen Wang, Xiao Qin, Chuan Lei, Zhengyuan Shen, Christos Faloutsos, Asterios Katsifodimos, George Karypis, Lijie Wen, and Philip S. Yu. 2023. Automatic Table Union Search with Tabular Representation Learning. In *Findings of the Association for Computational Linguistics: ACL 2023*. 3786–3800.

[32] Aamod Khatiwada, Grace Fan, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1 (2023), 1–25.

[33] Aamod Khatiwada, Harsha Kokel, Ibrahim Abdelaziz, Subhajit Chaudhury, Julian Dolby, Oktie Hassanzadeh, Zhenhan Huang, Tejaswini Pedapati, Horst Samulowitz, and Kavitha Srinivas. 2025. Tabsketchfm: Sketch-based tabular representation learning for data discovery over data lakes. In *ICDE*. 1523–1536.

[34] Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. 2022. Integrating Data Lake Tables. *Proc. VLDB Endow.* 16, 4 (2022), 932–945.

[35] Rohit Khoja, Devanshu Gupta, Yanjie Fu, Dan Roth, and Vivek Gupta. 2025. Weaver: Interweaving SQL and LLM for Table Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 28282–28308.

[36] Taehee Kim, Seungbin Yang, Jihwan Kim, and Jaegul Choo. 2026. Retrieve Only Relevant Tables Whether Few or Many: Adaptive Table Retrieval Method. In *Findings of the Association for Computational Linguistics: ACL 2026*. 13031–13054.

[37] Harsha Kokel, Aamod Khatiwada, Tejaswini Pedapati, Haritha Ananthakrishnan, Oktie Hassanzadeh, Horst Samulowitz, and Kavitha Srinivas. 2025. TOPJoin: A Context-Aware Multi-Criteria Approach for Joinable Column Search. In *VLDB Workshops (TaDA)*.

[38] Wojciech Kosiuk, Xingyu Ji, Yeounoh Chung, Fatma Özcan, and Madelon Hulsebos. 2026. Fine-Grained Table Retrieval Through the Lens of Complex Queries. In *Proceedings of the 11th Workshop on Automated Knowledge Base Construction (AKBC 2026)*. 94–107.

[39] Mayank Kothyari, Dhruva Dhingra, Sunita Sarawagi, and Soumen Chakrabarti. 2023. CRUSH4SQL: Collective Retrieval Using Schema Hallucination For Text2SQL. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14054–14066.

[40] Christos Koutras, Jiani Zhang, Xiao Qin, Chuan Lei, Vassilis N Ioannidis, Christos Faloutsos, George Karypis, and Asterios Katsifodimos. 2025. OmniMatch: Joinability discovery in data products. *Proc. VLDB Endow.* 18, 11 (2025), 4588–4601.

[41] Wonjin Lee, Kyumin Kim, Sungjae Lee, Jihun Lee, and Kwang In Kim. 2026. Piece of Table: A Divide-and-Conquer Approach for Selecting Subtables in Table Question Answering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 31672–31688.

[42] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 13067–13075.

[43] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.

[44] Xianming Li and Jing Li. 2023. AnglE-optimized Text Embeddings. *arXiv preprint arXiv:2309.12871* (2023).

[45] Feng Luo, Hai Lan, Hui Luo, Zhifeng Bao, Xiaoli Wang, J. Shane Culpepper, and Shazia Wasim Sadiq. 2026. Decomposition-Driven Multi-Table Retrieval and Reasoning for Numerical Question Answering. In *ICDE*. 1872–1885.

[46] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to information retrieval*. Cambridge University Press.

[47] Marc Maynou, Sergi Nadal, Raquel Panadero, Javier Flores, Oscar Romero, and Anna Queralt. 2026. FREYJA: Efficient join discovery in data lakes. *IEEE Trans. Knowl. Data Eng.* 38 (2026), 1–12.

[48] Md Mahadi Hasan Nahid and Davood Rafiei. 2024. TabSQLify: Enhancing Reasoning Capabilities of LLMs Through Table Decomposition. In *Proceedings of*

the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). 5725–5737.

[49] Fatemeh Nargesian, Erkang Zhu, Renée J. Miller, Ken Q. Pu, and Patricia C. Arocena. 2019. Data Lake Management: Challenges and Opportunities. *Proc. VLDB Endow.* 12, 12 (2019), 1986–1989.

[50] Fatemeh Nargesian, Erkang Zhu, Ken Q. Pu, and Renée J. Miller. 2018. Table Union Search on Open Data. *Proc. VLDB Endow.* 11, 7 (2018), 813–825.

[51] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming* 14, 1 (1978), 265–294.

[52] OpenAI. 2026. ChatGPT (GPT-5.4 mini model). https://chatgpt.com. Accessed: 2026-10-15.

[53] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *Advances in neural information processing systems* 36 (2023), 36339–36348.

[54] Aécio Santos, Aline Bessa, Fernando Chirigati, Christopher Musco, and Juliana Freire. 2021. Correlation sketches for approximate join-correlation queries. In *SIGMOD*. 1531–1544.

[55] Anish Das Sarma, Lujun Fang, Nitin Gupta, Alon Y. Halevy, Hongrae Lee, Fei Wu, Reynold Xin, and Cong Yu. 2012. Finding related tables. In *SIGMOD*. 817–828.

[56] Henrique Schechter Vera, Sahil Dua, et al. 2025. EmbeddingGemma: Powerful and Lightweight Text Representations. (2025). https://arxiv.org/abs/2509.20354

[57] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large Language Models Can Be Easily Distracted by Irrelevant Context. In *Proceedings of the 40th International Conference on Machine Learning*, Vol. 202. 31210–31227.

[58] Qi Shi, Han Cui, Haofeng Wang, Qingfu Zhu, Wanxiang Che, and Ting Liu. 2024. Exploring Hybrid Question Answering via Program-based Prompting. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11035–11046.

[59] Hassan Soliman, Vivek Gupta, Dan Roth, and Iryna Gurevych. 2026. CORE-T: COherent REtrieval of Tables for Text-to-SQL. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. 2082–2114.

[60] Yuan Sui, Jiaru Zou, Mengyu Zhou, Xinyi He, Lun Du, Shi Han, and Dongmei Zhang. 2024. TAP4LLM: Table Provider on Sampling, Augmenting, and Packing Semi-structured Data for Large Language Model Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 10306–10323.

[61] Chaojie Sun, Bin Cao, Tiantian Li, Chenyu Hou, Ruizhe Li, and Jing Fan. 2026. FGTR: Fine-Grained Multi-Table Retrieval via Hierarchical LLM Reasoning. *arXiv preprint arXiv:2603.12702* (2026).

[62] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[63] Haonan Wang, Jiaxiang Liu, Yurong Liu, Austin Senna Wijaya, Tianle Zhou, Eden Wu, Yijia Chen, Wanting You, Reya Vir, Daniela Pinto Veizaga, Grace Fan, Yusen Zhang, Juliana Freire, and Eugene Wu. 2026. LakeQA: An Exploratory QA Benchmark over a Million-Scale Data Lake. In *Proceedings of the International Conference on Machine Learning (ICML)*.

[64] Qiming Wang and Raul Castro Fernandez. 2023. Solo: Data Discovery Using Natural Language Questions Via A Self-Supervised Approach. *Proc. ACM Manag. Data* 1, 4 (2023), 1–27.

[65] Yuhui Wang, Jinqi Liu, Chengliang Chai, Hangyu Zhao, Yuhao Deng, Yuyu Luo, Xin Tang, Ye Yuan, Guoren Wang, Fengjin Wang, and Lei Cao. 2026. EcoTable: Cost-effective Table Integration in Data Lakes for Natural Language Queries. *Proc. VLDB Endow.* 19, 11 (2026), 2936–2949.

[66] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian M. Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2024. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. In *International Conference on Learning Representations*.

[67] Jian Wu, Linyi Yang, Dongyuan Li, Yuliang Ji, Manabu Okumura, and Yue Zhang. 2025. MMQA: Evaluating LLMs with multi-table multi-hop complex questions. In *The thirteenth international conference on learning representations*.

[68] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]

[69] An Yang, Baosong Yang, et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[70] Zhen Yang, Ziwei Du, Minghan Zhang, Wei Du, Haodong Zou, Shicheng Tan, Jie Chen, and Shu Zhao. 2026. SIP: Aligning LLM-based TableQA with Table Semantics via SQL Logic. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. 44287–44299.

[71] Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. 2023. Large Language Models are Versatile Decomposers: Decomposing Evidence and Questions for Table-based Reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 174–184.

[72] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 3911–3921.

[73] Xiaohan Yu, Pu Jian, and Chong Chen. 2025. TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 14063–14082.

[74] Jiani Zhang, Sercan Ö. Arık, Cosmin Arad, Fatma Özcan, and Alon Halevy. 2026. An Agentic Approach to Metadata Reasoning. *arXiv preprint arXiv:2604.20144* (2026).

[75] Xuanliang Zhang, Dingzirui Wang, Longxu Dou, Qingfu Zhu, and Wanxiang Che. 2025. MURRE: Multi-Hop Table Retrieval with Removal for Open-Domain Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*. 5789–5806.

[76] Yilun Zhao, Lyuhao Chen, Arman Cohan, and Chen Zhao. 2024. TaPERA: Enhancing Faithfulness and Interpretability in Long-Form Table QA by Content Planning and Execution-based Reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12824–12840.

[77] Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. MultiHiertt: Numerical Reasoning over Multi Hierarchical Tabular and Textual Data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6588–6600.

[78] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *SIGMOD*. 847–864.

[79] Erkang Zhu, Fatemeh Nargesian, Ken Q. Pu, and Renée J. Miller. 2016. LSH Ensemble: Internet-Scale Domain Search. *Proc. VLDB Endow.* 9, 12 (2016), 1185–1196.

[80] Moshé M. Zloof. 1977. Query-by-Example: A Data Base Language. *IBM Systems Journal* 16, 4 (1977), 324–343.

APPENDIX

A LLM PROMPTS

A.1 Profiling

Figure 8 shows the prompt we use to profile tables in the offline phase (described in Section 4.1). We prompt the LLM once per table to describe the table and each of its columns using the table name, column names, and first five rows. The LLM output then becomes the semantic profiles that are embedded into the column and table indexes. We use GPT-5.4-mini with temperature 0 by default, and we ablate the LLM in Section 5.5.

A.2 Answer-Table Sketches

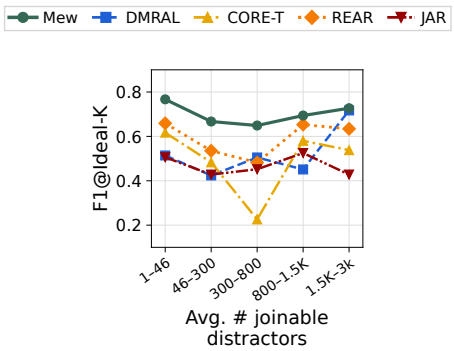
In the online phase (Section 4.2.1), Mew first prompts the LLM to create an answer-table sketch, given only the question, as a list of columns and a set of constraints (shown in Figure 9).

After retrieving candidate tables, Mew then prompts the LLM to ground the sketch using data lake tables, given the question, the current sketch, and the table name, columns, and first five rows of the highest-ranked candidates (shown in Figure 10). In a single call, the LLM selects a set of focus tables and refines the sketch with the column names and values of the data-lake candidates (described in Section 4.2.2).

A.3 Downstream Text-to-SQL Prompt

To measure downstream execution accuracy (Section 5.6), we prompt a text-to-SQL LLM with a question and the tables returned by each method. We use the same prompt for all methods (Figure 11), varying the LLM in Section 5.6. Following the prompts used in LLM-based text-to-SQL systems [59], the prompt includes each retrieved table's name, columns, and sample values, and instructs the LLM to generate a SQL query and infer join keys from matching column names and sample values.

B ROBUSTNESS



(a) SPIDER$_{\text{OPEN}}$

Figure 7: F1@Ideal-K on SPIDER$_{\text{OPEN}}$ for five groups of questions of equal size, ordered by the average number of joinable distractors of their gold tables.

We further analyze the robustness of Mew on SPIDER$_{\text{OPEN}}$ in Figure 7, similar to the analysis from Section 5.3 and Figure 5. As in Section 5.3, the joinable distractors of a gold table are the data lake tables that share at least one column name with it and are not gold tables of the question. We split the questions into five groups of equal size by the average number of joinable distractors of their gold tables. Similar to the analysis on BIRD$_{\text{OPEN}}$, Mew achieves the highest F1@Ideal-K in every group of questions. However, the gap between Mew and the baselines is smaller than on BIRD$_{\text{OPEN}}$, since SPIDER questions generally expect fewer tables in their gold set, with 61% of SPIDER questions requiring no join compared to 26% of BIRD questions. When a single table is expected to answer a question, retrieving the tables most similar to the question is often sufficient, which favors the baselines.

Semantic Profile Prompt (offline, per table)

System message: You are an expert in table question answering.

Input (filled per table):
`Table Name: {table_name}`
`Columns: {comma-separated column names}`
`Sample Values (first 5 rows): {first 5 rows of the table, as a Markdown table}`

Task: Describe the topic of the table (what this table is about). For each column, describe what kind of data it likely contains and its semantics. Provide concise descriptions that capture the key semantics of each column in relation to the table.

Output format: Return the output as a JSON object with the following format:
`{ "<table_name>": "table description", "<column_1>": "description of <column_1>", . . . , "<column_n>": "description of <column_n>" }`

Only return the JSON object, no additional text.

Figure 8: Prompt for the semantic profile of a table in the offline phase (Section 4.1).

Answer-Table Sketch Generation Prompt (online, per question)

System message: You are an expert in table question answering.

You are an expert at extracting table schemas from natural language questions.
Your task: Given a question, return information that describe an ideal table to answer it.

Output format (MUST follow exactly):

- ***Columns:*** A comma-separated list of column names in brackets. Example: `[ColumnA, ColumnB, ColumnC]`
- ***Constraints:*** Key-value pairs for column filters, entities, time. Example: `{Filters = 'ValueA', Entities = 'CategoryB', Time = '2020'}`.

IMPORTANT:

- Output ONLY the two lines above. No other text.
- Column names go in square brackets: `[col1, col2, col3]`
- Constraints are in curly braces: `{key1 = 'value1', key2 = 'value2'}`
- Include ALL explicit filters from the question as constraints.
- Keep concrete values verbatim whenever possible.
- Do NOT use generic placeholder constraint keys such as Filters, Entities, Time, Other, Condition, or Value.
- Prefer specific schema-like names over abstract categories.

Question: `{question}`

Figure 9: Prompt for generating the initial answer table sketch from the question (Section 4.2.1).

Sketch Refinement Prompt (online, combined focus-table selection and sketch refinement)

System message: You are an expert in table question answering.

You select relevant candidate tables AND refine an answer-table sketch for this question, in one pass, using data-lake candidates. The focus selection and the refined sketch will both be used to select the tables needed to answer this question, from a data lake of tables from many domains.

Question: {question}

Current answer-table sketch:
Columns: [{sketch columns}]
Constraints: {col_1 = 'value_1', col_2 = 'value_2', . . . }

Candidate table schemas: {the top-10 candidate tables, each rendered as a pipe-delimited table: a ### header with the table name, the column names, and the first 5 rows of the table, with each cell truncated to 60 characters}

Task 1 – Focus table selection:

- Select tables likely to contain answer-relevant attributes for the question entities/filters/time.
- Prefer entity and value tables over pure connector/bridge tables.
- Return only table names present in the candidate list.
- If a field is uncertain, return an empty list for that field.

Task 2 – Sketch refinement:

- Use the candidate table schemas and sample values to fix missing or incorrect columns and constraints.
- Keep schema-faithful column names and real constraints only.
- Treat your own Task 1 focus tables as soft priors for refinement, not hard facts.
- Add join keys or bridge columns only when the candidates make them necessary.
- Do not invent pseudo-columns or abstract constraint keys.
- Preserve semantic roles from the current sketch. Do NOT collapse distinct roles into one generic column.
- Avoid generic columns (type/status/value/amount/date/id/name/count/code) unless they are the only schema-faithful option.
- Do not output duplicate column names.
- Do not place multiple different '=' constraints on the same column unless candidate samples clearly support that exact logic.
- Keep high-specificity columns when available instead of replacing them with generic aliases.
- Use exact candidate column names when possible.
- If no change is needed, preserve the current sketch.

Output format (return exactly these three lines):
Focus Tables: [table1, table2, table3]
Columns: [col1, col2, col3]
Constraints: {col = 'value'}

Figure 10: Prompt for grounded refinement, which selects the focus tables and refines the sketch in a single call (Section 4.2.2).

Downstream NL2SQL Generation Prompt (per question)

System message: You are an expert SQL query generator. Return only valid SQL.

Input (filled per question):
Database Schema: {for each retrieved table: table name, column names + types, per-column top-5 value hints, and sample rows}
Question: {question}

Task: Given the database schema, sample data, and distinct sample rows, write a single SQLite SELECT query that answers the question, following these instructions:
1. Use ONLY the tables and columns provided in the schema above. Do not invent table or column names.
2. If the question requires joining tables, infer foreign keys from matching column names, the Table Relationships above (if given), and sample values. Do not join a table more than once.
3. Use the EXACT values from the database. Treat the [values: ...] hints as illustrative examples, not an exhaustive list.
4. Use double quotes around column names that contain spaces or special characters. Use single quotes for string literals – escape apostrophes by doubling them (e.g., 'Women"s Soccer').
5. For columns with cryptic names (like A1, A2, A3...), examine the sample values and value hints to determine which column matches the question's intent.
6. Use SQLite-compatible syntax (e.g., strftime('%Y', date_col) for year extraction, SUBSTR for substrings).
7. Ensure the SELECT clause is specific to the question (don't just SELECT *). Only select the columns needed to answer the question.
8. Return ONLY the SQL query. No explanation.

Output format: Return only the raw SQL query text, with no explanation, no markdown code fences, and no surrounding commentary.

Figure 11: Prompt for downstream text-to-SQL generation, shared by all methods (Section 5.6).